

FACULDADE DE TECNOLOGIA DE JACAREÍ

Relatório Técnico – Implementação e Testabilidade

Disciplina: Engenharia de Software II

Professor: André Olímpio

Aluno: Felipe Pacheco

Data de entrega: 14/11/2025

Base: Estudo de caso e enunciado da atividade (implementação e testabilidade).

Fonte do enunciado: Atividade - Reposição de Aula (08/11/2025).

1. Introdução

Este relatório descreve a proposta de implementação e as estratégias de testabilidade para o sistema

web de agendamento de consultas médicas, com base no levantamento de requisitos e na modelagem

prévia. O objetivo é apresentar a pilha tecnológica, a organização do código, as boas práticas adotadas,

os tipos de testes planejados e exemplos de casos de teste ligados aos requisitos. A abordagem prioriza

qualidade, manutenibilidade e automação dos testes.

2. Descrição da Implementação

2.1 Linguagens e Tecnologias Sugeridas

Pilha recomendada (foco em TypeScript): - Backend: Node.js com TypeScript, framework Express ou

NestJS. - ORM: Prisma ou TypeORM. - Banco de dados: PostgreSQL. - Frontend: React (TypeScript) ou

Next.js (opcional). - Contêineres: Docker + Docker Compose (ambientes: dev, test, prod). - CI/CD: GitHub

Actions (ou GitLab CI) para automatizar builds e testes. - Testes: Jest (unit/integration), Supertest (API

integration), Cypress (E2E/functional). - Observabilidade: Sentry (erros), Prometheus/Grafana (métricas) e

logs estruturados.

2.2 Estrutura básica do código e organização em camadas

Sugestão de organização (camadas): 1) api/ (controllers/routes) — rotas HTTP e validações (DTOs). 2)

services/ — lógica de negócio, regras e orquestração. 3) repositories/ (data access) — interação com o

banco via ORM (prisma/typeORM). 4) models/ (domínio) — entidades, tipos e interfaces.
5) tests/ —

casos de teste (unit, integration, e2e). 6) config/ — configuração de ambientes, variáveis de ambiente e

leitura segura (.env). 7) infra/ — inicialização de banco, filas, serviços externos (ex.: notificador). Essa

separação favorece a testabilidade (camadas com responsabilidades claras), a reutilização e a

manutenção.

2.3 Boas práticas aplicadas

- Separação de responsabilidades (SoC) e princípios SOLID. - Injeção de dependência para facilitar

mocking em testes. - Uso de DTOs e validação (Zod / class-validator) para dados de entrada. -

Tratamento centralizado de erros e uso de middleware para logging. - Versionamento de API (v1) e

documentação com OpenAPI/Swagger. - Scripts de lint (ESLint) e formatação (Prettier) no pipeline. -

Testes automatizados com cobertura mínima definida (ex.: $\geq 80\%$).

3. Estratégias de Testabilidade

3.1 Importância da testabilidade

Testabilidade é a capacidade do sistema de ser testado de forma eficaz e eficiente. Ela reduz defeitos,

facilita refatorações e garante confiança na entrega contínua. A arquitetura e as práticas de

implementação devem facilitar a escrita de testes isolados e integrados.

3.2 Tipos de testes planejados

- Unitários: testar funções e serviços isolados. Ex.: validação de regra de negócio ao criar uma consulta.

- Integração: testar integração entre camadas (ex.: controllers -> services -> repositories) com banco de

dados em memória ou testcontainers.

- Funcionais / E2E: testar fluxos de usuário completos (ex.: agendamento pelo paciente) com Cypress ou

Playwright.

- Testes de contrato: garantir compatibilidade de APIs entre front e back (Pact ou Postman tests).

3.3 Exemplos de casos de teste (mapeados aos requisitos)

Requisito	Exemplo de Caso de Teste
RF01 - Cadastro de pacientes	Unit: validação de CPF e campos obrigatórios. Integration: POST /patients cria paciente.
RF03 - Agendamento de consultas	Unit: regra que impede agendar em horário passado. Integration: tentativa de agendar horário inválido.
RF04 - Cancelamento/remarcação	Integration: cancelar consulta altera status para 'cancelada' e libera vaga na agenda.
RF05 - Notificação	Integration: serviço de notificação enfileira mensagem; Mock confirma envio.

3.4 Ferramentas e frameworks recomendados

- Unit/Integration: Jest + ts-jest, Supertest (API). - E2E/Functional: Cypress ou Playwright.

-

Testcontainers: rodar PostgreSQL temporário durante testes de integração. - Mocking: jest.mock / sinon. -

Quality gates: SonarQube, coverage report (Istanbul/nyc). - CI: GitHub Actions com job de 'test'.

4. Integração entre Implementação e Testes

4.1 Arquitetura que facilita a testabilidade

Arquiteturas com separação clara entre controllers, services e repositories facilitam a inserção de testes.

Padrões como Hexagonal/Ports & Adapters e Injeção de Dependência permitem substituir implementações reais por doubles/mocks nos testes.

4.2 Automação de testes e CI/CD

Recomenda-se pipeline com os seguintes estágios: 1) lint + build (garante qualidade estática) 2) unit tests

(rápidos) 3) integration tests (com Testcontainers) 4) e2e tests (em staging, não obrigatórios em cada

push) Definir policy: pull requests só podem ser mesclados se build e testes unit/integration passarem e

cobertura mínima for atingida.

4.3 Benefícios para manutenção futura

- Testes automatizados reduzem regressões e permitem refatorações com segurança.
- Documentam o comportamento esperado.
- Automatização acelera feedback e melhora a qualidade das entregas.

5. Exemplos práticos de testes (pseudocódigo / snippets)

Exemplo de teste unitário (Jest): `test('não deve agendar em horário já ocupado', async () => { ... })`

Exemplo de teste de integração (Supertest + testcontainer): `describe('POST /appointments', () => { ... })`

Exemplo de teste E2E (Cypress): `cy.visit('/login');` ...

6. Análise Crítica

A testabilidade impõe escolhas arquiteturais (ex.: desacoplamento, DI). Em sistemas legados, adaptar

arquitetura pode demandar esforço inicial, mas os ganhos a médio e longo prazo justificam o

investimento.

7. Conclusão

Implementação planejada e testabilidade são componentes essenciais para entregar um sistema

confiável. Adotar separação de camadas, injeção de dependência, testes automatizados e pipelines de CI

garante entregas mais seguras e sustentáveis.